

Software Performance Engineering

Recitation 1.4

Atharva Kirkole & Sophia Sun
Thursday, Sep 24, 2026



Some due dates

- Homework 3 is released, due Oct 5.
- Project 1 beta is due 26th September!
- Submit your code to the GitHub repo
- **PLEASE SUBMIT EACH QUESTION IN SEPERATE PAGE** for Write Ups on GradeScope - Much easier to grade.
- Reach tier 20 to get at least a B grade for beta.
- No number guarantees A - it relies on various factors like code, tier, regular commits etc., Please try to get the tier as high as possible based on materials of class

Beta Writeup

- A brief overview of your design.
- The general state of completeness and expected performance of your implementation.
- Any additional information that you feel would be helpful to the staff in understanding your submission.
- An acknowledgment of any help received from course staff, classmates on Piazza, or publicly available materials + AI for appropriate uses only.

Optimization tips

- What we covered in recitations so far:
 - recitation 1.1: from bit rotation to block rotation
 - recitation 1.2: in-block rotation (row-column-row algorithm)
 - recitation 1.3: divide-and-conquer approach for RCR
 - Bit hacks
 - Bently rules:
 - Caching
 - Precomputation
 - Sub-expression elimination
 - Loop unrolling
 - Function inlining
- And so many more!

VECTORIZATION



Basic Vector Instructions

- Vector Register Sizes: xmm: 128, ymm: 256, ~~zmm: 512 bits~~
- Packing: Fit many values in one register
 - xmm fits 2x uint64_t, 4x int32_t, 4x float, etc...
 - in general for xmm = $(128/8) / \text{sizeof}(T)$
- Packed Arithmetic Operations
 - add, subtract, multiply, divide, reciprocate, max, min, sqrt, reciprocal of sqrt
- Packed Comparison Operations
- Packed Logical Operations
- Packed Type Conversions
 - int to float etc

Overlapping Memory Regions

- When `A[]` and `B[]` overlap, `memcpy(B, A, n)` can be wrong
 - because the `memcpy` can overwrite data from the source, which it would need to use later
- Same thing can happen with vector operations
- To vectorize codes that deal with two arrays, need to know they don't overlap
- We can mark this using the `restrict` keyword
- `int* restrict A, int* restrict B`
 - The only way to obtain pointers from A's memory region is by offsetting from A
 - Thus A and B cannot overlap because there are no (non UB) offsets into each other

What can a compiler auto-vectorize?

- “Pure” (not interdependent) loop iterations
 - Dependence between loop iterations(example: NO $a[i] = a[i-1] + x$)
- Reductions
 - Sum or product of contiguous memory: can detect the result variable is used for reduction
- Inductions
 - $A[i] = i$;
- If Statements
 - Certain if statements can be converted to branchless code(MASK)
- Stride
 - $A[i] += B[i * 4]$;
- When restrict is not known
 - Generates both vector and scalar code paths
 - Tests for overlap at runtime

Associativity of Reductions

- $1 + (2 + 3) == (1 + 2) + 3$
- $0.1 + (0.2 + 0.3) != (0.1 + 0.2) + 0.3$
- Floating point addition is not associative. This means order matters.
- Vector operations do not add the values in the same order as a scalar loop.
- -Compiler has to match O0 semantics
- So, at -O3 it refuses to vectorize float reductions
- For float: Need the -ffast-math flag to tell the compiler we're ok with this.

USEFUL INTRINSICS



Alignment Hint

```
void *__builtin_assume_aligned(const void *arg,  
size_t align);
```

Returns its first argument and allows the compiler to assume that the returned pointer is at least align bytes aligned.

```
// x is at least 16 byte aligned
```

```
void *x = __builtin_assume_aligned(arg, 16);
```

```
// (char *) x - 8 is 32 byte aligned (offset by 8)
```

```
void *x = __builtin_assume_aligned(arg, 32, 8);
```

Other Bit Manipulations

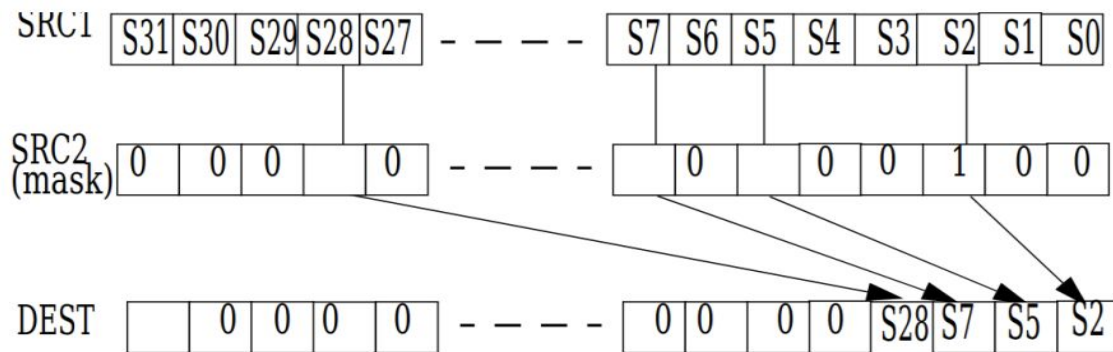
```
uint64_t __builtin_rotateleft64(uint64_t x, bits_t  
amt);  
uint64_t __builtin_rotateright64(uint64_t x, bits_t  
amt);  
//within byte  
uint64_t __builtin_bitreverse64(uint64_t x);  
// image is stored MSB-is-leftmost, but x86 is little-endian, reading 8 bytes as a uint64_t reverses their order  
uint64_t __builtin_bswap64(uint64_t x); //AA-88->88-AA  
  
bits_t __builtin_popcountll(uint64_t x);  
bits_t __builtin_clzll(uint64_t x); // from MSBs  
bits_t __builtin_ctzll(uint64_t x); //from LSBs
```

Parallel Bit Extraction

```
#include <immintrin.h>
```

```
uint64_t _pext_u64(uint64_t s1, uint64_t mask);
```

Transfer either contiguous or non-contiguous bits in the first source operand to contiguous low order bit positions in the destination according to the mask values.



Packed Shift

```
#include <immintrin.h>
__m128i _mm_sll_epi64(__m128i reg, __m128i
count);
```

Shift the two 64 bit numbers packed in reg left by count.

Pre Fetching!

```
__builtin_prefetch (const void *addr[, rw[,  
locality]])
```

Takes:

- Addr to prefetch from
- Read mode or write mode
- Locality (L1 /L2/ L3/ auto)

load $i+(x)$ along with i itself so that that $i+x$ th index is already loaded in by the time our execution reaches it.

x = how many fetches can be in progress at once.

Intel Intrinsic Guide

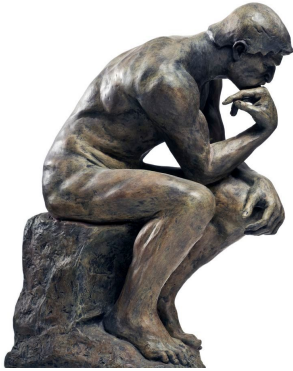
[Intel® Intrinsic Guide](#)

PERFORMANCE ENGINEERING



Performance Engineering

think,



code,



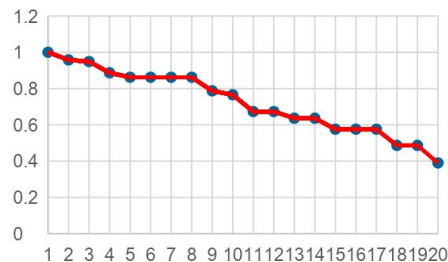
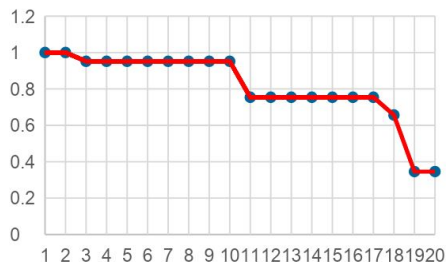
run,

measure,



Basic Performance-Engineering Workflow

1. Measure the performance of Program A.
2. Make a change to Program A to produce a hopefully faster Program A'.
3. Measure the performance of Program A'.
4. If A' beats A, set $A = A'$.
5. If A is still not fast enough, go to Step 2.



If you can't measure performance **reliably**,
it is hard to make many small changes that add up

Example: Timing a Code for Sorting

```
#include <stdio.h>
#include <time.h>

void my_sort(double *A, int n);
void fill(double *A, int n);

int main() {
    int max = 4 * 1000 * 1000;
    int min = 1;
    int step = 20 * 1000;
    double A[max];
    struct timespec start, end;

    for (int n=min; n<max; n+=step) {
        fill(A, n);

        clock_gettime(CLOCK_MONOTONIC, &start);
        my_sort(A, n);
        clock_gettime(CLOCK_MONOTONIC, &end);

        double tdiff = (end.tv_sec -
            start.tv_sec)
            + 1e-9*(end.tv_nsec -
            start.tv_nsec);
        printf("size %d, time %f\n", n, tdiff);
    }
    return 0;
}
```

Library for `clock_gettime()`

Sorting routine to be timed.

Auxiliary routine for filling array with random numbers.

Used by `clock_gettime()`:

```
struct timespec {
    time_t tv_sec; /* seconds */
    long tv_nsec; /* nanoseconds */
};
```

Inspired by a study
due to Sivan Toledo.

Example: Timing a Code for Sorting

```
#include <stdio.h>
#include <time.h>

void my_sort(double *A, int n);
void fill(double *A, int n);

int main() {
    int max = 4 * 1000 * 1000;
    int min = 1;
    int step = 20 * 1000;
    double A[max];
    struct timespec start, end;

    for (int n=min; n<max; n+=step) {
        fill(A, n);

        clock_gettime(CLOCK_MONOTONIC, &start);
        my_sort(A, n);
        clock_gettime(CLOCK_MONOTONIC, &end);

        double tdiff = (end.tv_sec -
                        start.tv_sec)
                        + 1e-9*(end.tv_nsec -
                        start.tv_nsec);
        printf("size %d, time %f\n", n, tdiff);
    }
    return 0;
}
```

Loop over arrays of increasing length.

Array randomly filled.

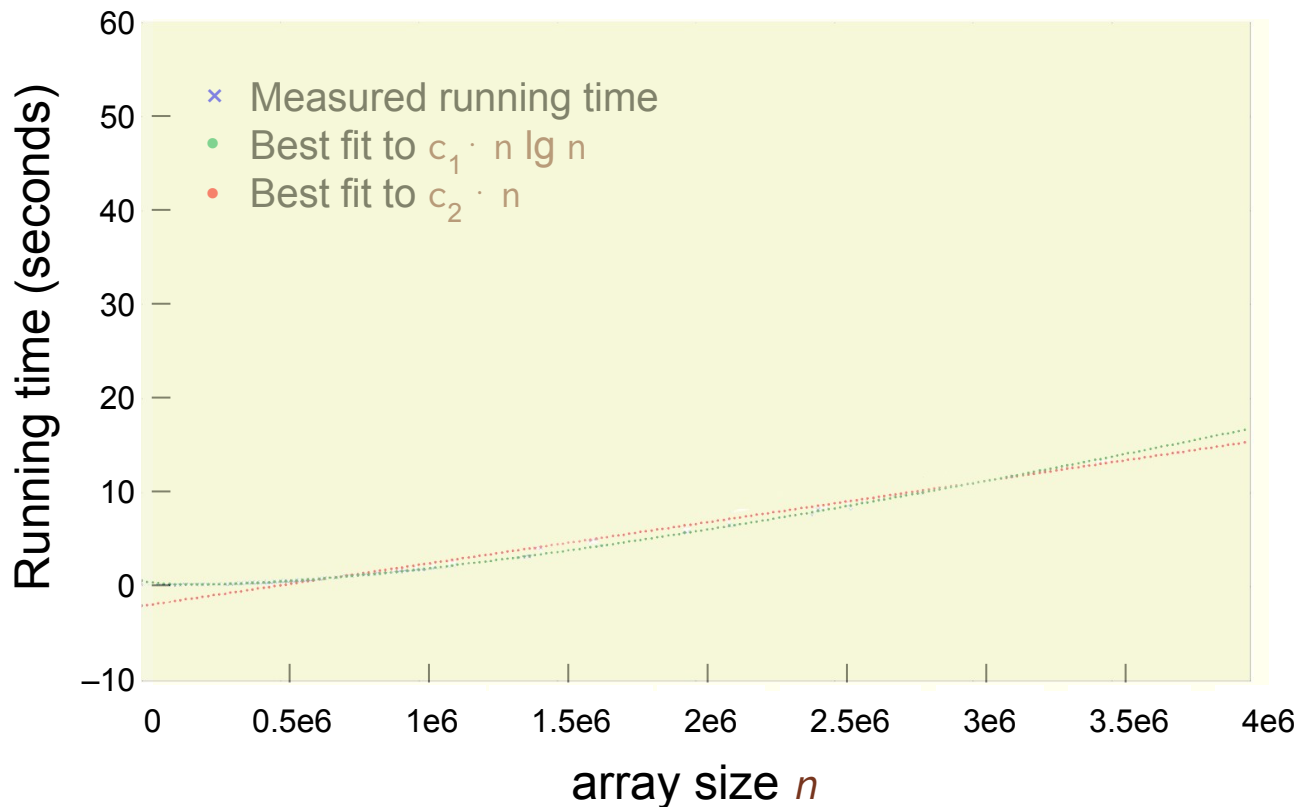
Measure time before sorting.

Sort.

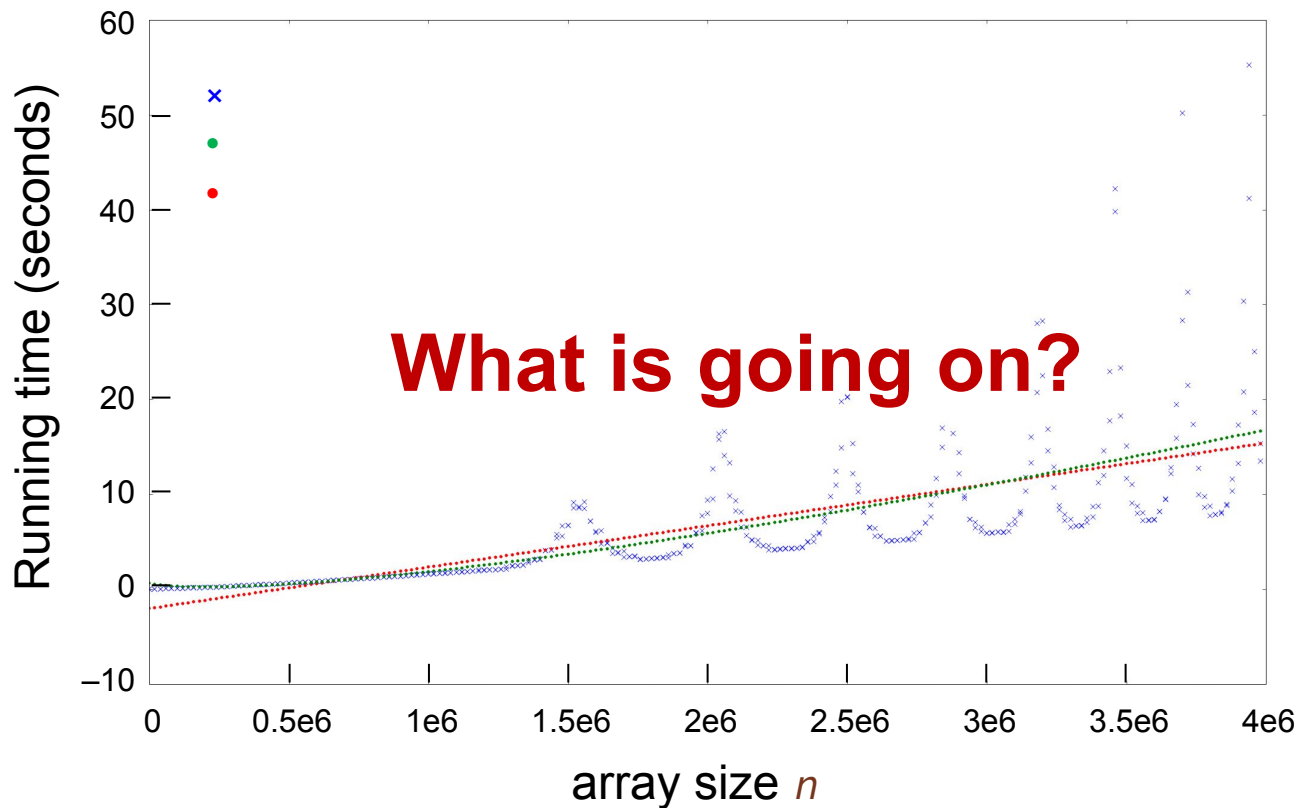
Measure time after sorting.

Compute elapsed time.

Running Times for Sorting



Running Times for Sorting



Dynamic Voltage and Frequency Scaling

DVFS is a technique to dynamically trade power for performance by **adjusting** the **clock frequency** and **supply voltage** to transistors

- Reduce operating frequency if chip is too hot or otherwise to save power.
- Reduce voltage if frequency is reduced.
- **Turbo Boost** increases frequency if the chip is cool.

Dynamic Voltage and Frequency Scaling

DVFS is a technique to dynamically trade power for performance by **adjusting** the **clock frequency** and **supply voltage** to transistors

- Reduce operating frequency if chip is too hot or otherwise to save power.
- Reduce voltage if frequency is reduced.
- **Turbo Boost** increases frequency if the chip is cool.

$$\text{Power} \propto CV^2f$$

C = dynamic capacitance $\approx$ roughly area $\times$ activity (how many bits toggle)

V = supply voltage

f = clock frequency

Changing frequency and voltage has a cubic effect on power (and heat)

Today's Topic

How can one **reliably** measure the performance of software?



We'll start with **What Statistics and Metrics To Measure**

WHAT STATISTICS AND METRICS TO MEASURE



Summary Statistics and Noise

Suppose that you measure the performance of a **deterministic program 100 times*** with the same input on a computer with some **interfering background noise**.

What statistic best represents the raw performance of the software?

- ☐ mean
- ☐ median
- ☐ maximum
- ☐ minimum

* we start it cold 100 times to eliminate any kind of caching effect from previous runs

Summary Statistics and Noise

Suppose that you measure the performance of a **deterministic program 100 times*** with the same input on a computer with some **interfering background noise**.

What statistic best represents the raw performance of the software?

- ☐ mean
- ☐ median
- ☐ maximum
- ☒ minimum

Minimum does the best at **noise rejection**, because we expect that any measurements higher than the minimum are due to noise.

Summarizing Ratios

Trial	Program A	Program A'
1	9	3
2	8	2
3	2	20
4	10	2
Mean	7.25	6.75

Summarizing Ratios

Trial	Program A	Program A'	A/A'
1	9	3	3.00
2	8	2	4.00
3	2	20	0.10
4	10	2	5.00
Mean	7.25	6.75	3.03

Conclusion

Program A' is > 3 times better than A

WRONG!

Turn the Comparison Upside-Down

Trial	Program A	Program A'	A/A'	A'/A
1	9	3	3.00	0.33
2	8	2	4.00	0.25
3	2	20	0.10	10.00
4	10	2	5.00	0.20
Mean	7.25	6.75	3.03	2.70

Paradox

A' is 3.03x faster than A & A is 2.70x faster than A'

Observation

The arithmetic mean of A/A' is **NOT** the inverse of the arithmetic mean of A'/A

Geometric Mean

Trial	Program A	Program A'	A/A'	A'/A
1	9	3	3.00	0.33
2	8	2	4.00	0.25
3	2	20	0.10	10.00
4	10	2	5.00	0.20
Mean	(a) 7.25	(a) 6.75	(g) 1.57	(g) 0.64

Formula

$$\left(\prod_{i=1}^n a_i \right)^{1/n} = \sqrt[n]{a_1 a_2 \cdots a_n}$$

Observation

The geometric mean of A/A' **is** the inverse of the geometric mean of A'/A

Selecting among Summary Statistics

Given server, service as many requests as possible

- Arithmetic mean
- CPU utilization

Most cloud service requests are satisfied within 100 ms

- 90th percentile
- Wall-clock time

Best game-playing AI

- Arithmetic mean
- Win rate

Fit into a machine with 100 MB of memory

- Maximum
- Memory use

Support frequent use on a mobile device

- Arithmetic mean
- Energy use or CPU utilization

Most environmentally friendly

- Arithmetic mean
- Carbon footprint

Meet a customer service-level agreement (SLA)

- Weighted combo of statistics
- Multiple metrics

CODING TIME BEFORE BETA DUE!



Optimization tips

- Always take a shot if you have an idea!
- Decide for yourself: what works for others may not work for you (even an idea from a course staff!)
 - a course staff's suggestion is also not the only solution.
- It's not guaranteed that an optimization idea will actually make your code run faster.
 - if that happens, think about why.
- Take one step at a time to make debugging easier.
- Write comments, so you won't forget what you're doing or planning to do a week later.

6 recitations for project 1

1. 4-way rotation
2. introduction to rcr, image walkthrough, naive col rotation
3. bit hacks review, divide-and-conquer using bit masks (find the excel table)
4. Measurement1, Q&A session before beta due
5. vectorization, built-in intrinsics (after beta submission)
6. the queen's problem using bit hacks (homework 4)
7. tiling, prefetching, performance gap table: technique, tier, engineering time estimation